# EasyNet Ontology and CLI Layering

Consensus Specification (pre-implementation draft)

Silan Hu
National University of Singapore

Draft, April 6, 2026

**Abstract**

This document records the consensus reached between the EasyNet authors and a rapporteur during a multi-round design alignment session, regarding the ontology that underpins EasyNet—an agent-native distributed system—and the layering of its command-line interface. The central claim aligned upon is that an *ability* is not a function and not a static contract, but rather the triple "*skill plus self-evolution plus network-wide invocation*," implemented internally as a graph composed of memory and workflow components. We further pin down the object-oriented view of agents (public abilities, private skills), the two-language two-runtime split (AAL/BCC for production; EAL/EasyNet for consumption), the demotion of *device* from a network-first-class object to a hosting substrate, and the desugaring of `agent send` into a single-line External EAL mission invoking the target's default `chat` ability. Each statement in the document is explicitly tagged as an axiom (directly stated by the principals), a derivation (logically deduced from one or more axioms), or a deferred item (known gap, deliberately postponed). The intent of this artifact is to lock the model before any implementation work begins, so that the upcoming refactor and all future epics share the same vocabulary.

## Contents

**1 Reading guide** **2**

**2 The two core technical theses** **2**
2.1 Thesis A: Agent Ability . . . 2
2.2 Thesis B: EasyNet, the network of vocational abilities . . . 3

**3 Self-evolution as graph** **3**
3.1 Internal structure of an evolving ability . . . 3
3.2 What "self-evolution" actually means . . . 4
3.3 What distillation operates on . . . 4

**4 The OOP view (load-bearing)** **4**
4.1 Agent as object, ability as public method, skill as private field . . . 4
4.2 Visibility rules . . . 4
4.3 Five questions collapsed into one axiom . . . 5
4.4 Encapsulation invariant . . . 5

**5 Two languages, two runtimes** **5**
5.1 The pairing . . . 5
5.2 Producer side versus consumer side . . . 5
5.3 Two layers of EAL . . . 6
5.4 Where the current `easynet` binary fits . . . 6

**6 Device is a hosting substrate (interpretation C) 6**
6.1 Device is not a network-first-class entity . . . 6
6.2 What this means concretely . . . 7
6.3 Logical identity versus physical placement . . . 7
6.4 Cardinality . . . 7
6.5 Known transitional misalignment . . . 7

**7 Default abilities and call sugar 8**
7.1 `chat` as the default callable . . . 8
7.2 `agent send` is sugar for the default ability . . . 8

**8 Six locked-in design decisions 8**

**9 CLI surface (consensus shape) 9**
9.1 Top-level groups . . . 9
9.2 Backwards compatibility . . . 10
9.3 Deliberately absent commands . . . 10

**10 Non-CLI artifacts shipped in this PR 10**
10.1 The `ability_graph_traces` field on mission run metadata . . . 10
10.2 Retention of `agent_sessions.rs` . . . 11

**11 Three time scales 11**

**12 Higher-level judgment and next deliverables 11**

**13 Open recursion points 12**

**14 Conclusion 12**

# 1 Reading guide

Every substantive statement in this document carries one of three tags:

**[axiom]** Directly stated by Silan, Sean, or in the alignment session transcript. Not negotiable in this document. If you wish to revise an axiom, a separate alignment round is required.

**[derived]** Logically derived from one or more axioms. Negotiable: if you disagree with a derivation, the disagreement should be raised *before* implementation begins, and either the derivation is revised or the axioms it depends on are clarified.

**[deferred]** A known gap, deliberately left for a future epic. The document explicitly identifies these so that downstream design work does not silently fill them with the wrong assumption.

The tagging discipline is the most important property of this document. It makes the difference between "what we agreed" and "what I inferred" auditable.

# 2 The two core technical theses

## 2.1 Thesis A: Agent Ability

**[axiom]**

> **ability = skill + self-evolution + network-wide invocation**
>
> Compared with the static-template execution of a Skill, an Ability can continuously evolve through user long-context and network compounding, and through EasyNet can remotely invoke abilities across the whole network for its own use, achieving maximum capability execution.

The three components of an ability are non-optional. Each contributes a property that the other two cannot supply on their own:

- **+ skill** — the seed. The ability begins as wrapped skill knowledge.
- **+ self-evolution** — the ability's *implementation* grows over time, while its public surface remains stable.
- **+ network-wide invocation** — the ability is addressable from anywhere on EasyNet via gRPC.

[derived] Removing any one component degenerates the object back to something strictly simpler:

- without graph ⇒ degenerates to a skill,
- without self-evolution ⇒ degenerates to a frozen function,
- without network invocation ⇒ degenerates to a local tool.

### 2.2 Thesis B: EasyNet, the network of vocational abilities

[axiom]

> **EasyNet is the communication-and-collaboration network for network-wide vocational abilities.**

Two interaction modalities live on EasyNet:

- **Communication** (light interaction): post, feed, chat, connection.
- **Collaboration** (deep interaction): functional execution, the actual cross-agent ability calls.

[derived] The CLI under design here is concerned almost exclusively with the *collaboration* modality. The communication modality is a separate surface (most likely UI / social), and is intentionally out of scope for this document.

## 3 Self-evolution as graph

### 3.1 Internal structure of an evolving ability

[axiom] The core of self-evolution is a *graph*, which is split into *memory* and *workflow*:

```
Ability implementation = graph
  |-- memory graph   (knowledge / long context / accumulated state)
  |-- workflow graph   (execution structure / decision flow / call topology)
```

### 3.2 What "self-evolution" actually means

[derived]

- The *public surface* of an ability (its signature) does not evolve freely; that would break callers.
- The *graph* behind the surface evolves continuously as new memory accrues (from long context, traces, and user feedback) and as new workflow paths are discovered (better orderings, new sub-ability calls).
- An ability is therefore *the same ability* even as its internal graph grows. Distillation operates on the *graph*, not on the surface signature.

### 3.3 What distillation operates on

[derived]

Distillation only ever distills **abilities**, never skills.

The reason is observability: skills are private to a single agent and have no network trace; abilities are public and have invocation traces from across the network. Trace data feeds back into the graph (memory growth and workflow refinement). It does not feed back into skills.

## 4 The OOP view (load-bearing)

### 4.1 Agent as object, ability as public method, skill as private field

[axiom] An agent class in AAL takes the following shape:

```
class Agent {
    public:
        ability chat(context) -> reply
        ability review(code) -> review
        ability decompose(goal) -> [subgoals]

    private:
        skill thinking_style
        skill code_review_prior
        skill memory_compression_strategy
        skill ...
}
```

[axiom] `chat` and `review` are typical *default abilities* every agent exposes, analogous to `Object.toString()` in Java.

### 4.2 Visibility rules

[axiom]

- **Ability is public.** It has a frozen signature, is discoverable network-wide, and is callable from External EAL.
- **Skill is private.** It has no signature on the network, is not discoverable from outside the agent, and is *never* callable from External EAL.
- **EAL (external) can only write `agent.ability(...)`.** It cannot reach into skills.

- **Public calls go over gRPC.** That is what makes "network-wide invocation" real.

### 4.3 Five questions collapsed into one axiom

The visibility rule simultaneously answers five design questions that, before the alignment session, appeared independent:

| Question | Answer (because of visibility rule) |
|---|---|
| Why does ability need a graph? | Because it must self-evolve; static structure cannot grow. |
| Why is skill private? | Because it is single-agent internal knowledge; sharing makes no sense across agent boundaries. |
| Why gRPC? | Because ability is a network service. |
| Why does the EAL compiler only see ability? | Because it operates in `public` scope. |
| Why does distillation only distill abilities? | Because only public calls produce observable network traces. |

[derived] These five answers are now consequences of one rule, not five independent design decisions. This collapse is the central simplification of the alignment session.

### 4.4 Encapsulation invariant

**Invariant**

No CLI command, no SDK call, and no EAL construct may reach across an agent boundary into a skill. If a future need arises ("I want my agent to learn from another agent's skill"), the only valid answer is: *the other agent must wrap that skill as an ability.*

[derived] This invariant is the load-bearing wall of the system. Violating it collapses the OOP model and, with it, the ability/skill distinction. Every proposed CLI verb in this document is required to pass an encapsulation check before being added.

## 5 Two languages, two runtimes

### 5.1 The pairing

[axiom] EasyNet has two distinct language/runtime pairs:

| Language | Interpreter | Expresses |
|---|---|---|
| AAL (Agent Abstraction Language) | BCC (Brain Cell Coding) | how an agent class is defined |
| EAL (EasyNet Ability Language) | EasyNet | how abilities compose to accomplish a task |

BCC interprets AAL. EasyNet interprets EAL. **They are two independent language/runtime pairs**, not a single compilation pipeline.

### 5.2 Producer side versus consumer side

[derived]

- AAL/BCC is the *producer side*: who builds an agent, what abilities it exposes.
- EAL/EasyNet is the *consumer side*: who orchestrates agents to accomplish a task.
- The bridge between them is the *gRPC ability endpoint*: BCC exposes abilities as gRPC services; EasyNet calls them via gRPC.

[derived] This separation means: **writing a new agent does not require knowing EAL; writing a new mission does not require knowing AAL.** This is clean separation of concerns and is one of the strongest properties of the design.

### 5.3 Two layers of EAL

EAL appears in two distinct scopes:

| | **External EAL** | **Internal EAL** |
|---|---|---|
| Who writes | Humans, in `*.eal` files | Generated by the ability graph at call time |
| Scope | `public` — only sees abilities | `private` — sees the agent's own skills, memory, abilities |
| Enforced by | EasyNet's EAL parser at compile time | The agent's own runtime |
| Persisted as | Mission runs | Per-call execution traces inside an ability invocation |
| OOP analogue | Application code calling public methods | The body of a method, which can touch private fields |

[derived] The two layers use the same surface language but operate under different access scopes, exactly as Java syntax compiles differently inside versus outside a class definition.

### 5.4 Where the current `easynet` binary fits

[derived] The current `easynet` binary is the *local CLI surface for the EAL runtime* (i.e. EasyNet itself). It does not yet contain BCC or any AAL machinery. Anywhere this document refers to "the EasyNet runtime," the local artifact in question is the `easynet` binary plus its embedded `axon` runtime process.

[deferred] BCC will arrive as a separate concern. When it does, the `easynet` binary may grow a sibling (`bcc`) or absorb BCC as a subcommand. That decision is future work.

## 6 Device is a hosting substrate (interpretation C)

### 6.1 Device is not a network-first-class entity

[axiom]

> Device is **not** a first-class network object. It is the physical substrate on which abilities are deployed — like an OS to a process.

Three interpretations of "device" were considered during the alignment session:

(A) Device = degenerate Agent (an agent without any skill). *Rejected*: pulls device back into network identity.

(B) Device = a parallel specialization of Process, peer to Agent. *Rejected*: same problem.

(C) Device = hosting substrate, not first-class on the network. **Chosen.**

### 6.2 What this means concretely

[derived] The set of network-first-class objects collapses to two:

| Network first-class | Host-local, not first-class |
|---|---|
| Agent (the only addressable actor) | Device (physical substrate, hardware, credentials) |
| Ability (public method endpoint) | Axon runtime (the OS-level hosting process) |
| | MCP server (local stdio process for AI clients) |

[derived] The network only sees **agents talking to agents via abilities**. Devices exist, but only as deployment metadata. They never appear as the "to" of an ability call from outside.

### 6.3 Logical identity versus physical placement

[derived]

- **Logical identity** (network-visible): an agent is identified by `<tenant>/<agent-name>`, e.g. `silan/reviewer`. This is what appears in `agent.ability(...)` calls.
- **Physical placement** (hidden): which device currently hosts that agent is metadata, accessible only through `device` CLI commands and only for diagnostic purposes.

The current PR uses `<tenant>/<agent-name>` as the minimum viable form. A future extension to `<tenant>/<agent-name>#<instance-id>` is planned for when multiple instances of the same agent class need to coexist.

### 6.4 Cardinality

[derived] Because device is not first-class:

- One device can host multiple agent instances.
- One agent instance can in principle span multiple devices (if its memory graph and workflow runtime live on different machines), and the network still sees a single agent.

[derived] This is the affordance interpretation C buys us that A and B do not. It *leaves room for distributed agents*, even though the current implementation does not yet exercise it.

### 6.5 Known transitional misalignment

**Transitional misalignment**

The current Axon SDK exposes `deploy_package(node_id, ...)` — ability deployment is currently routed by *device node id*, not by agent logical id. Under the new ontology this is a leak: the public API should be `deploy_package(agent_id, ...)`, with the runtime resolving which device hosts that agent.

[deferred] This is **not fixed in the current PR.** The CLI command `easynet ability deploy <path> --to <node>` retains its current shape. Its long-help text explicitly notes this is a transitional form.

# 7 Default abilities and call sugar

## 7.1 `chat` as the default callable

[axiom] Every agent class typically exposes a `chat(prompt) -> reply` method as a "default callable." This is analogous to:

- `Object.toString()` in Java,
- `__call__` in Python,
- `operator()` in C++,
- the `()` of a Functor in Haskell.

## 7.2 `agent send` is sugar for the default ability

[axiom]

> "实际上 `send` 是默认交给 `agent.chat()` ability, 只是默认语法糖。" ("In fact, `send` dispatches to the `agent.chat()` ability by default; it is merely default syntactic sugar.")

Two layers of sugar are at play:

```
# Layer 1 (CLI sugar):
easynet agent send claude "hello"
  desugars to a single-line External EAL mission:
let r = claude.chat(prompt: "hello")
print(r)

# Layer 2 (EAL sugar, future):
let r = claude.chat(prompt: "hello")
  may be writable as:
let r = claude("hello")
  because chat is the agent's default callable
```

[derived] Three consequences follow:

1. **`agent send` is not a special command.** It is the shortest possible mission. This unifies "send to one agent" and "run a mission" under one execution model.
2. **Agents are callable objects.** The default `chat` ability gives every agent a built-in call operator. Custom agents can override or supplement it.
3. **Mission is the universal execution model.** Once `agent send` is sugar for a one-line mission, every cross-agent interaction in the system goes through the mission runtime. There is no "second path."

# 8 Six locked-in design decisions

These were resolved during the alignment session and are not reopened by this document. They represent the minimum set of decisions required to begin implementation.

| # | Question | Decision | Rationale |
|---|---|---|---|
| 1 | Agent instance ID format | `<tenant>/<agent-name>`, with `#<instance-id>` reserved for the future | Logical, network-visible, decoupled from device. |
| 2 | Agent instantiation in current PR | Not done. Agents remain implicit, provisioned by runtime plus deploy. | AAL/BCC do not exist yet; premature commands would lock in wrong semantics. |
| 3 | `device join` semantics | Registers a hosting substrate (physical device plus credentials). | Matches the Docker-daemon-register / Kubernetes-node-join pattern under interpretation C. |
| 4 | `agent send` semantics | Sugar for a single-line External EAL mission invoking the target's default `chat` ability. | Unifies all cross-agent interaction under the mission execution model. |
| 5 | `discuss` / `think` placement | Primary location is `mission` (`mission discuss`, `mission think`). Deprecated alias retained under `agent` for backwards compatibility. | Both are mission patterns (multi-agent orchestration; iterative planning loop), not agent-instance methods. |
| 6 | `device config` versus `runtime config` | Semantically `runtime config`; physically remains `device config` for now. Long-help explicitly notes the migration. | Settings (exec enable, MCP behavior) control the runtime, not the physical device. Code migration deferred to avoid CLI churn. |

# 9 CLI surface (consensus shape)

## 9.1 Top-level groups

```
easynet
|
|-- agent       Manage agent instances (network actors)
|     |-- add / list / remove / doctor       (instance lifecycle)
|     |-- send                                (sugar for chat)
|     |-- session new/list/show/append/end    (memory dimension)
|     |-- discuss          DEPRECATED -> use `mission discuss`
|     `-- think            DEPRECATED -> use `mission think`
|
|-- device      Manage hosting substrates (physical machines)
|     |-- join / reset / config               (this host's lifecycle)
|     `-- list / show / remove                (substrates on the network)
|         (no rename / tag: would require an admin ability that doesn't exist)
|
|-- ability     Manage public ability endpoints
|     |-- list / show                          (discovery)
|     |-- deploy / uninstall                   (lifecycle; deploy is host-routed)
|     |-- invoke                               (direct call)
|     `-- exec                                 (one-shot ad-hoc command)
|         (no `update`: conflates three time scales)
|         (no `logs`: wrong artefact)
```

```
|
|-- mission     Run External EAL programs and patterns
|     |-- compile / run / list / show / cancel
|     |-- discuss                              (multi-agent orchestration)
|     `-- think                                (iterative planning loop)
|
|-- runtime     Local Axon process lifecycle
|     `-- start / stop / status / connect / logs
|
|-- mcp         Local MCP server process
|     `-- serve / status
|
|-- doctor      Aggregated health check                          (NEW top-level)
`-- completion  Shell completion generator                       (NEW top-level)
```

### 9.2 Backwards compatibility

All current top-level commands (`start`, `stop`, `status`, `connect`, `devices`, `abilities`, `deploy`, `invoke`, `exec`, `mission`, `join`, `reset`, `config`, `mcp-server`, `mcp-install`, `agent`, `discuss`, `skill-install`, `think`) remain available as **deprecated aliases** that:

- continue to function identically,
- emit a one-line stderr deprecation notice pointing at the new layered command,
- will be removed in a future release (no date set).

This makes the current PR a **purely additive, zero-breakage** change.

### 9.3 Deliberately absent commands

The following commands are missing *on purpose*. Adding them would violate the ontology:

- `easynet skill <anything>` — skills are private; exposing them breaks the encapsulation invariant of §4.
- `easynet capability <anything>` — capability is the cross-process invocation abstraction; introducing it half-baked would lock in premature semantics.
- `easynet ability update` — conflates three distinct time scales (signature bump, graph evolution, policy retraining). See §11.
- `easynet ability logs` — the real artefact is the ability graph trace, which lives at a layer the current PR does not yet model.
- `easynet agent spawn` — agent instantiation is an AAL/BCC concern; nothing in the current PR can honestly spawn an agent class instance.

## 10 Non-CLI artifacts shipped in this PR

### 10.1 The `ability_graph_traces` field on mission run metadata

A new optional field is added to `MissionRunMeta`:

```
/// Per-cross-agent-ability-call execution summaries. Each entry
/// captures what the target agent's ability graph did to satisfy
/// one call. Empty for runs that only invoked device abilities
```

```
/// (which have no graph).
///
/// Schema is intentionally Value here: this is a landing slot for
/// the upcoming ability-graph trace format, not the format itself.
#[serde(default, skip_serializing_if = "Option::is_none")]
pub ability_graph_traces: Option<Vec<serde_json::Value>>,
```

[derived] Section 3 states that ability implementations are graphs. The mission run metadata is the natural landing spot for traces of those graphs, even before the trace format is fully specified. Naming the field `ability_graph_traces` (instead of, e.g., `internal_eal_summaries` or `dispatch_traces`) *teaches the next reader* that an ability has a graph, by the field name alone.

### 10.2 Retention of `agent_sessions.rs`

The previously-considered deletion of this file is reversed. Sessions are the simplest form of the *memory* dimension of an agent (per-caller conversation history). They live on the agent side of the OOP wall, are private by default, and are properly modeled as "the calling client's view of one slice of an agent's memory."

## 11 Three time scales

When evaluating any future CLI command, ask which time scale it operates on:

| Time scale | Frequency | Example operation | Allowed verb |
|---|---|---|---|
| Schema/SLA bump | discrete, human-in-loop | publishing v2 of an ability with a new input field | `ability deploy` (new version) |
| Graph evolution | low-frequency, semi-automatic | a new memory pattern accrues; a workflow path is shortened | none yet (internal) |
| Per-call execution | realtime | one ability call picks a route through the graph and executes | `ability invoke`, `mission run` |

[derived] Verbs that try to compress two scales into one (for example `ability update` covering both v-bump and graph tweak) **must be rejected**. They look convenient but corrupt the user's mental model.

## 12 Higher-level judgment and next deliverables

[axiom] (Silan): the next deliverable beyond this document should be a paper-level **System Model: Definition + Invariants + Diagram**, suitable for a whitepaper or top-tier venue framing. This is the system's true moat.

The current PR explicitly *does not* attempt to be that paper. It implements the operational consequences of the consensus reached above. The paper-level model is tracked as the next epic. A short pointer to that deliverable will appear at the end of `docs/ARCHITECTURE.md`:

**Note**

**Next:** `docs/SYSTEM_MODEL.md` (paper-level formalization). Until that exists, this document and `ARCHITECTURE.md` are the source of truth for ontological questions about EasyNet.

## 13 Open recursion points

These are honestly identified gaps. They are not blockers for the current PR, but they are blockers for the next epic.

1. **Capability abstraction.** "Capability" as the unifying invocation interface across agent and device is mentioned but not specified. This is the next ontological extension.

2. **Internal EAL trace format.** §5.3 states that internal EAL is persisted as per-call execution traces, but the schema is not specified. The `ability_graph_traces` field lands the slot but defers the format.

3. **Default ability set.** §7 says `chat` is a typical default. Whether `chat` is *the* default, whether `review` is also default, and whether agents must declare a default — all unspecified.

4. **Agent class system.** AAL is named, but its grammar, type system, and inheritance model (if any) are entirely unspecified.

5. **Tenant model.** `<tenant>/<agent-name>` is chosen as the ID format but tenant lifecycle, sharing, and authorization are unspecified.

6. **Status of `mcp-install` and `skill-install`.** Whether these legacy top-level commands should remain, move under `agent install`, or be removed entirely depends on whether skill-install should exist at all under interpretation C. They are kept as legacy aliases in the current PR with the question explicitly flagged.

## 14 Conclusion

This document closes a multi-round alignment session by recording, with auditable provenance, what the principals agreed and what the rapporteur inferred. The single most consequential agreement is the OOP visibility rule of §4, which by itself collapses five previously-open design questions (§4.3) into a single axiom. Around it, the two-language two-runtime split (§5) and the demotion of device to a hosting substrate (§6) provide the remaining structure required to begin implementation without anticipating the next ontological extension.

The implementation that follows this document is, by design, the smallest change that can reflect the new ontology in the CLI surface. Anything that would require the next epic — capability abstraction, AAL/BCC, the formal system model — is deliberately deferred and explicitly enumerated in §13, so that no future contributor will be forced to guess which concepts are settled and which are still open.

---

*This document is the consensus output of the alignment session and should be reviewed in full before any code is written. If you disagree with a **[derived]** statement, raise it before implementation begins. If you disagree with an **[axiom]**, a separate alignment round is required.*